



GUIDA COMPLETA E CODICE SORGENTE: ONLINE SHOP APP



Fase 1: Inizializzazione e Configurazione dell'Ambiente

1. **Creazione del progetto** con il template ufficiale TypeScript di Expo:

```
npx create-expo-app@latest OnlineShopAppTS --template blank-typescript
```

2. **Spostarsi nella cartella** appena creata:

```
cd OnlineShopAppTS
```

Installazione delle dipendenze della navigazione

1. Installa i pacchetti base di React Navigation

```
npm install @react-navigation/native @react-navigation/stack
```

2. Lascia che Expo installi le sue componenti native compatibili

```
npx expo install react-native-screens react-native-safe-area-context
```

(oppure forzando la risoluzione dei pacchetti e usando Expo per i moduli nativi, evitando conflitti):

```
npm install @react-navigation/native @react-navigation/stack --legacy-peer-deps  
npx expo install react-native-screens react-native-safe-area-context --fix
```



Fase 2: Struttura delle Cartelle

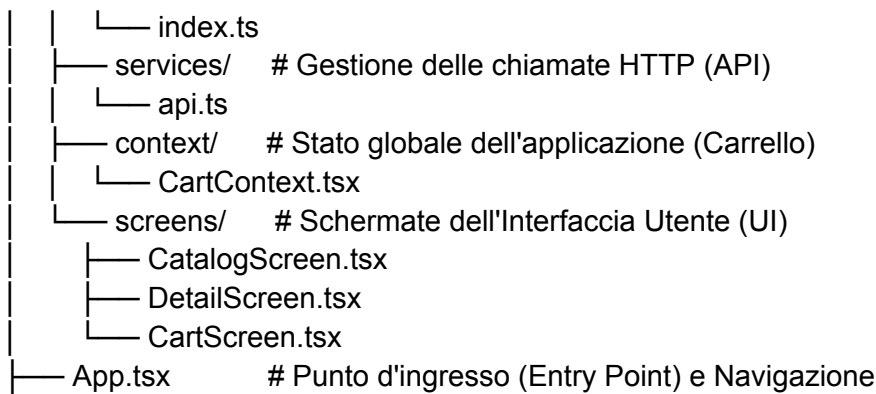
All'interno della cartella principale del progetto, crea la cartella **src** e le relative sottocartelle in modo da organizzare il codice secondo il principio della **Separazione delle Responsabilità (Separation of Concerns)**:

Plaintext

OnlineShopAppTS/

| — src/

| — types/ # Modelli di dato (Interfacce TypeScript)



Fase 3: Codice Sorgente dei File

1. Definizione dei Tipi Dati

File: `src/types/index.ts`

Questo file definisce i contratti dei dati. TypeScript bloccherà la compilazione se proviamo a usare proprietà inesistenti.

TypeScript

// Rappresenta l'oggetto Sotto-Categoria associato a ogni prodotto

```
export interface Category {
  id: number;
  name: string;
  image: string;
}
```

// Rappresenta la struttura principale del Prodotto restituito dall'API

```
export interface Product {
  id: number;
  title: string;
  price: number;
  description: string;
  images: string[];
  category: Category;
}
```

// Rappresenta l'elemento salvato nel carrello: il prodotto unito alla sua quantità locale

```
export interface CartItem {
  product: Product;
  quantity: number;
}
```

2. Strato dei Servizi API (Repository)

File: `src/services/api.ts`

Questo codice è blindato: include un filtro di sicurezza che scarta i dati corrotti immessi dagli utenti esterni nel database di Platzi ed esclude URL non validi.

TypeScript

```
import { Product } from '../types';

// URL Ufficiale della Fake API di Platzi
const BASE_URL = 'https://api.escuelajs.co/api/v1';

// Recupera la lista dei primi 30 prodotti dal server
export const getProducts = async (): Promise<Product[]> => {
  const response = await fetch(`${BASE_URL}/products?offset=0&limit=30`);

  // Se la risposta HTTP non è nel range 200-299, lancia un errore intercettato dal
  // componente
  if (!response.ok) throw new Error('Impossibile caricare il catalogo prodotti');

  const data: any[] = await response.json();

  // FILTRO DI SICUREZZA BLINDATO: Rimuove i prodotti con dati nulli, mancanti o URL
  // immagini fake
  return data.filter(item =>
    item &&
    typeof item.id === 'number' &&
    typeof item.title === 'string' &&
    typeof item.price === 'number' &&
    item.category &&
    typeof item.category.name === 'string' &&
    Array.isArray(item.images) &&
    item.images.length > 0 &&
    typeof item.images[0] === 'string' &&
    item.images[0].startsWith('http') // Accetta solo link HTTP reali ed esistenti
  );
};

// Recupera i dettagli completi di un singolo prodotto partendo dal suo ID numerico
export const getProductDetails = async (id: number): Promise<Product> => {
  const response = await fetch(`${BASE_URL}/products/${id}`);
  if (!response.ok) throw new Error('Dettagli del prodotto non trovati sul server');
  return await response.json();
};
```

3. Stato Globale dell'Applicazione (Context API)

File: `src/context/CartContext.tsx`

Gestisce la logica di business del carrello (aggiunta, rimozione, svuotamento e calcolo dei totali in tempo reale) rendendola accessibile a qualsiasi schermata.

TypeScript

```
import React, { createContext, useState, useContext, ReactNode } from 'react';
import { Product, CartItem } from '../types';
```

```
// Dichiarazione esplicita di tutto ciò che il Context esporrà alle schermate UI
```

```
interface CartContextType {
  cartItems: CartItem[];
  addToCart: (product: Product) => void;
  removeFromCart: (productId: number) => void;
  clearCart: () => void;
  totalItemsCount: number;
  totalPrice: number;
}
```

```
// Inizializzazione del contesto (valore di partenza undefined, gestito poi dal Provider)
```

```
const CartContext = createContext<CartContextType | undefined>(undefined);
```

```
// Provider Component: Avvolge l'intera applicazione per iniettare lo stato
```

```
export const CartProvider = ({ children }: { children: ReactNode }) => {
  const [cartItems, setCartItems] = useState<CartItem[]>([]);
```

```
// Aggiunge un prodotto al carrello o ne incrementa la quantità se già presente
```

```
const addToCart = (product: Product) => {
  setCartItems((prevItems) => {
    const existingItem = prevItems.find((item) => item.product.id === product.id);
    if (existingItem) {
      // Se esiste già, mappiamo l'array incrementando solo la quantità dell'elemento mirato
      return prevItems.map((item) =>
        item.product.id === product.id
          ? { ...item, quantity: item.quantity + 1 }
          : item
      );
    }
    // Se è un nuovo prodotto, lo aggiungiamo all'array impostando la quantità iniziale a 1
    return [...prevItems, { product, quantity: 1 }];
  });
};
```

```
// Riduce la quantità di un prodotto o lo rimuove del tutto se scende a zero
```

```
const removeFromCart = (productId: number) => {
  setCartItems((prevItems) => {
    const existingItem = prevItems.find((item) => item.product.id === productId);
    if (!existingItem) return prevItems;

    if (existingItem.quantity === 1) {
```

```

    // Rimuove l'elemento dall'elenco filtrandolo via
    return prevItems.filter((item) => item.product.id !== productId);
  }
  // Riduce la quantità di 1 unità
  return prevItems.map((item) =>
    item.product.id === productId
      ? { ...item, quantity: item.quantity - 1 }
      : item
  );
});
};

// Resetta completamente il carrello svuotando l'array lo stato
const clearCart = () => setCartItems([]);

// Calcolo automatico reattivo del numero totale di pezzi nel carrello
const totalItemsCount = cartItems.reduce((sum, item) => sum + item.quantity, 0);

// Calcolo automatico reattivo del prezzo totale complessivo
const totalPrice = cartItems.reduce((sum, item) => sum + item.product.price * item.quantity, 0);

return (
  <CartContext.Provider value={{ cartItems, addToCart, removeFromCart, clearCart,
    totalItemsCount, totalPrice }}>
    {children}
  </CartContext.Provider>
);
};

// Hook personalizzato (Custom Hook) per utilizzare il carrello nelle schermate con controllo
di sicurezza
export const useCart = () => {
  const context = useContext(CartContext);
  if (!context) throw new Error('useCart deve essere invocato obbligatoriamente dentro un
  CartProvider');
  return context;
};

```

4. Schermata: Catalogo Prodotti

File: `src/screens/CatalogScreen.tsx`

Mostra l'elenco dei prodotti filtrati. Include la Safe Area per sollevare la barra inferiore sopra i tasti di navigazione fisici o virtuali dello smartphone.

TypeScript

```

import React, { useEffect, useState } from 'react';
import { View, Text, FlatList, Image, TouchableOpacity, ActivityIndicator, StyleSheet } from
'react-native';
import { SafeAreaView } from 'react-native-safe-area-context'; // Protezione layout per
notch/tasti virtuali
import { getProducts } from '../services/api';
import { useCart } from '../context/CartContext';
import { Product } from '../types';

// Funzione correttiva: Rimuove le parentesi quadre e virgolette annidate restituite per errore
dal database di Platzi
const cleanImageUrl = (url: string): string => {
  if (!url) return 'https://via.placeholder.com/150';
  let cleaned = url.replace(/[\[\]]/g, "");
  if (!cleaned.startsWith('http')) return 'https://via.placeholder.com/150';
  return cleaned;
};

export default function CatalogScreen({ navigation }: { navigation: any }) {
  const [products, setProducts] = useState<Product[]>([]);
  const [loading, setLoading] = useState<boolean>(true);
  const [error, setError] = useState<string | null>(null);
  const { totalItemsCount, totalPrice } = useCart();

  // Effetto eseguito al primo caricamento della schermata per scaricare i dati
  useEffect(() => {
    getProducts()
      .then((data) => {
        setProducts(data);
        setLoading(false);
      })
      .catch((err) => {
        setError(err.message);
        setLoading(false);
      });
  }, []);

  // Interfaccia di caricamento (Spinner)
  if (loading) {
    return (
      <View style={styles.center}>
        <ActivityIndicator size="large" color="#4F46E5" />
        <Text style={{ marginTop: 10 }}>Caricamento catalogo Platzi...</Text>
      </View>
    );
  }

  // Interfaccia in caso di errore di connessione

```

```

if (error) {
  return (
    <View style={styles.center}>
      <Text style={styles.errorText}> ⚠ {error}</Text>
    </View>
  );
}

return (
  // edges bottom-left-right evita il doppio spazio bianco in alto coordinandosi con la navbar
  dello Stack
  <SafeAreaView style={styles.container} edges={['bottom', 'left', 'right']>
    <FlatList
      data={products}
      keyExtractor={({item}) => item.id.toString()}
      renderItem={({item}) => (
        <TouchableOpacity
          style={styles.productCard}
          onPress={() => navigation.navigate('Detail', { productId: item.id })}
        >
          <Image
            source={{ uri: cleanImageUrl(item.images[0]) }}
            style={styles.productImage}
          />
          <View style={styles.productInfo}>
            { /* RISOLTO: Accediamo a .name dell'oggetto categoria senza stamparlo intero */ }
            <Text style={styles.category}>{item.category.name}</Text>
            <Text style={styles.title} numberOfLines={2}>{item.title}</Text>
            <Text style={styles.price}>€ {item.price.toFixed(2)}</Text>
          </View>
        </TouchableOpacity>
      )}
    />

    { /* Barra riassuntiva del carrello posizionata in basso in modo fisso */ }
    <View style={styles.cartBar}>
      <View>
        <Text style={styles.cartBarText}>{totalItemsCount} prodotti nel carrello</Text>
        <Text style={styles.cartBarSub}>Totale stimato: € {totalPrice.toFixed(2)}</Text>
      </View>
      <TouchableOpacity style={styles.cartButton} onPress={() =>
        navigation.navigate('Cart')>
        <Text style={styles.cartButtonText}>Vai al carrello</Text>
      </TouchableOpacity>
    </View>
  </SafeAreaView>
);
}

```

```

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#F9FAFB' },
  center: { flex: 1, justifyContent: 'center', alignItems: 'center', padding: 20 },
  errorText: { color: 'red', fontSize: 16, textAlign: 'center', fontWeight: 'bold' },
  productCard: { flexDirection: 'row', padding: 12, marginHorizontal: 16, marginVertical: 8,
    backgroundColor: '#FFF', borderRadius: 12, elevation: 2, shadowColor: '#000',
    shadowOffset: { width: 0, height: 1 }, shadowOpacity: 0.2, shadowRadius: 1.41 },
  productImage: { width: 90, height: 90, borderRadius: 8, backgroundColor: '#E5E7EB' },
  productInfo: { flex: 1, marginLeft: 12, justifyContent: 'center' },
  category: { fontSize: 12, color: '#6B7280', fontWeight: '600', textTransform: 'uppercase' },
  title: { fontSize: 15, fontWeight: 'bold', marginVertical: 4, color: '#1F2937' },
  price: { fontSize: 14, color: '#4F46E5', fontWeight: '700' },
  cartBar: { flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center', padding:
    16, backgroundColor: '#FFF', borderTopWidth: 1, borderColor: '#E5E7EB' },
  cartBarText: { fontWeight: 'bold', fontSize: 14, color: '#1F2937' },
  cartBarSub: { fontSize: 12, color: '#6B7280' },
  cartButton: { backgroundColor: '#4F46E5', paddingVertical: 10, paddingHorizontal: 16,
    borderRadius: 8 },
  cartButtonText: { color: '#FFF', fontWeight: 'bold' }
});

```

5. Schermata: Dettaglio Prodotto

File: `src/screens/DetailScreen.tsx`

Mostra le informazioni estese del singolo prodotto selezionato tramite ID inviato come parametro di navigazione.

TypeScript

```

import React, { useEffect, useState } from 'react';
import { View, Text, Image, StyleSheet, TouchableOpacity, ActivityIndicator, ScrollView } from
'react-native';
import { getProductDetails } from '../services/api';
import { useCart } from '../context/CartContext';
import { Product } from '../types';

const cleanImageUrl = (url: string): string => {
  if (!url) return 'https://via.placeholder.com/300';
  let cleaned = url.replace(/[\[\]]/g, '');
  if (!cleaned.startsWith('http')) return 'https://via.placeholder.com/300';
  return cleaned;
};

export default function DetailScreen({ route, navigation }: { route: any, navigation: any }) {
  // Estrazione dell'ID passato dal catalogo tramite i parametri della rotta
  const { productId } = route.params;

```



```

const [product, setProduct] = useState<Product | null>(null);
const [loading, setLoading] = useState<boolean>(true);
const [error, setError] = useState<string | null>(null);
const { addToCart } = useCart();

useEffect(() => {
  getProductDetails(Number(productId))
    .then((data) => {
      setProduct(data);
      setLoading(false);
    })
    .catch((err) => {
      setError('Impossibile caricare i dettagli di questo prodotto.');
```

setLoading(false);

```

    });
  }, [productId]);

if (loading) {
  return (
    <View style={styles.center}>
      <ActivityIndicator size="large" color="#4F46E5" />
    </View>
  );
}

if (error || !product) {
  return (
    <View style={styles.center}>
      <Text style={{ color: 'red', marginBottom: 20 }}> ⚠ {error}</Text>
      <TouchableOpacity style={styles.backButton} onPress={() => navigation.goBack()}>
        <Text style={styles.backButtonText}>Torna al catalogo</Text>
      </TouchableOpacity>
    </View>
  );
}

return (
  <ScrollView style={styles.container}>
    <Image
      source={{ uri: cleanImageUrl(product.images[0]) }}
      style={styles.largeImage}
    />
    <View style={styles.infoContainer}>
      { /* Nullish coalescing operator di sicurezza in caso la categoria sia opzionale */ }
      <Text style={styles.category}>{product.category?.name || 'Generico'}</Text>
      <Text style={styles.title}>{product.title}</Text>
      <Text style={styles.price}>€ {product.price.toFixed(2)}</Text>
      <Text style={styles.description}>{product.description}</Text>
    </View>
  </ScrollView>
);

```

```

    { /* Pulsante d'azione collegato alla logica globale del Context */
    <TouchableOpacity style={styles.addButton} onPress={() => addToCart(product)}>
      <Text style={styles.addButtonText}>🛒 Aggiungi al carrello</Text>
    </TouchableOpacity>

    <TouchableOpacity style={styles.backButton} onPress={() => navigation.goBack()}>
      <Text style={styles.backButtonText}>Indietro</Text>
    </TouchableOpacity>
  </View>
</ScrollView>
);
}

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#FFF' },
  center: { flex: 1, justifyContent: 'center', alignItems: 'center', padding: 20 },
  largeImage: { width: '100%', height: 320, resizeMode: 'cover', backgroundColor: '#E5E7EB' },
  infoContainer: { padding: 20 },
  category: { fontSize: 13, color: '#6B7280', fontWeight: '600', textTransform: 'uppercase',
  letterSpacing: 1 },
  title: { fontSize: 22, fontWeight: 'bold', marginVertical: 8, color: '#1F2937' },
  price: { fontSize: 20, color: '#4F46E5', fontWeight: '700', marginBottom: 16 },
  description: { fontSize: 15, color: '#4B5563', lineHeight: 22, marginBottom: 24 },
  addButton: { backgroundColor: '#4F46E5', padding: 16, borderRadius: 12, alignItems:
'center', marginBottom: 12 },
  addButtonText: { color: '#FFF', fontSize: 16, fontWeight: 'bold' },
  backButton: { borderWidth: 1, borderColor: '#D1D5DB', padding: 16, borderRadius: 12,
alignItems: 'center', width: '100%' },
  backButtonText: { color: '#4B5563', fontSize: 16, fontWeight: '600' }
});

```

6. Schermata: Il Carrello Locale

File: `src/screens/CartScreen.tsx`

Mostra l'elenco dei prodotti inseriti, calcola i subtotali dinamici per ogni riga e simula la chiusura dell'ordine pulendo lo stato.

TypeScript

```

import React from 'react';
import { View, Text, FlatList, StyleSheet, TouchableOpacity, Alert } from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import { useCart } from '../context/CartContext';

export default function CartScreen({ navigation }: { navigation: any }) {

```

```
// Consumiamo lo stato e tutte le funzioni esposte dal nostro hook personalizzato
const { cartItems, addToCart, removeFromCart, clearCart, totalItemsCount, totalPrice } =
useCart();
```

```
// Gestore per la simulazione dell'acquisto richiesto dalla traccia
const handleConfirmOrder = () => {
  Alert.alert("Ordine Confermato!", "Simulazione d'acquisto completata con successo.");
  clearCart(); // Svuota lo stato locale
  navigation.navigate('Catalog'); // Reindirizza l'utente alla home
};
```

```
return (
  <SafeAreaView style={styles.container} edges={['bottom', 'left', 'right']}>
    {cartItems.length === 0 ? (
      <View style={styles.center}>
        <Text style={styles.emptyText}>Il tuo carrello è vuoto.</Text>
      </View>
    ) : (
      <FlatList
        data={cartItems}
        keyExtractor={({item}) => item.product.id.toString()}
        renderItem={({ item }) => (
          <View style={styles.itemCard}>
            <View style={{ flex: 1 }}>
              <Text style={styles.itemTitle}>{item.product.title}</Text>
              <Text style={styles.itemPrice}>Unitario: € {item.product.price.toFixed(2)}</Text>
              <Text style={styles.subtotal}>
                Subtotale: € {(item.product.price * item.quantity).toFixed(2)}
              </Text>
            </View>
            { /* Sezione Selettore Quantità con pulsanti + e - */ }
            <View style={styles.quantityContainer}>
              <TouchableOpacity style={styles.qtyButton} onPress={() =>
removeFromCart(item.product.id)}>
                <Text style={styles.qtyText}>-</Text>
              </TouchableOpacity>
              <Text style={styles.quantityNumber}>{item.quantity}</Text>
              <TouchableOpacity style={styles.qtyButton} onPress={() =>
addToCart(item.product)}>
                <Text style={styles.qtyText}>+</Text>
              </TouchableOpacity>
            </View>
          </View>
        )}
      </FlatList>
    )}
  </SafeAreaView>
)

{ /* Footer fisso contenente i riepiloghi dei totali e i pulsanti d'azione */ }
```

```

<View style={styles.footer}>
  <View style={styles.totalRow}>
    <Text style={styles.totalLabel}>Numero totale prodotti:</Text>
    <Text style={styles.totalValue}>{totalItemsCount}</Text>
  </View>
  <View style={styles.totalRow}>
    <Text style={styles.totalLabel}>Totale complessivo:</Text>
    <Text style={styles.totalValue}>€ {totalPrice.toFixed(2)}</Text>
  </View>

  <TouchableOpacity
    style={[styles.actionButton, { backgroundColor: '#10B981' }]}
    onPress={handleConfirmOrder}
    disabled={cartItems.length === 0} // Disabilita il click se il carrello è vuoto
  >
    <Text style={styles.btnText}>Conferma ordine</Text>
  </TouchableOpacity>

  <TouchableOpacity
    style={[styles.actionButton, { backgroundColor: '#EF4444', marginVertical: 8 }]}
    onPress={clearCart}
    disabled={cartItems.length === 0}
  >
    <Text style={styles.btnText}>Svuota carrello</Text>
  </TouchableOpacity>

  <TouchableOpacity style={styles.backButton} onPress={() => navigation.goBack()}>
    <Text style={styles.backButtonText}>Indietro</Text>
  </TouchableOpacity>
</View>
</SafeAreaView>
);
}

```

```

const styles = StyleSheet.create({
  container: { flex: 1, backgroundColor: '#F9FAFB' },
  center: { flex: 1, justifyContent: 'center', alignItems: 'center' },
  emptyText: { fontSize: 16, color: '#6B7280', fontWeight: '500' },
  itemCard: { flexDirection: 'row', justifyContent: 'space-between', alignItems: 'center',
padding: 16, backgroundColor: '#FFF', marginHorizontal: 16, marginVertical: 6,
borderRadius: 12, elevation: 1 },
  itemTitle: { fontSize: 16, fontWeight: 'bold', color: '#1F2937' },
  itemPrice: { fontSize: 12, color: '#6B7280', marginVertical: 2 },
  subtotal: { fontSize: 14, color: '#4F46E5', fontWeight: '600' },
  quantityContainer: { flexDirection: 'row', alignItems: 'center' },
  qtyButton: { backgroundColor: '#E5E7EB', width: 32, height: 32, borderRadius: 16,
justifyContent: 'center', alignItems: 'center' },
  qtyText: { fontSize: 18, fontWeight: 'bold', color: '#1F2937' },

```

```

    quantityNumber: { marginHorizontal: 12, fontSize: 16, fontWeight: 'bold', color: '#1F2937' },
    footer: { padding: 20, backgroundColor: '#FFF', borderTopWidth: 1, borderColor: '#E5E7EB'
  },
  totalRow: { flexDirection: 'row', justifyContent: 'space-between', marginBottom: 8 },
  totalLabel: { fontSize: 14, color: '#4B5563' },
  totalValue: { fontSize: 16, fontWeight: 'bold', color: '#1F2937' },
  actionButton: { padding: 14, borderRadius: 10, alignItems: 'center' },
  btnText: { color: '#FFF', fontWeight: 'bold', fontSize: 16 },
  backButton: { alignItems: 'center', padding: 10 },
  backButtonText: { color: '#4B5563', fontSize: 14, fontWeight: '600' }
});

```

7. Entry Point del Progetto e Navigazione

File: `App.tsx`

Questo è il file principale (nella root del progetto). Monta il Navigatore Stack, definisce le rotte ammesse, tipizza rigorosamente i parametri e silenzia il warning interno di InteractionManager.

TypeScript

```

import React from 'react';
import { LogBox } from 'react-native';
import { NavigationContainer } from '@react-navigation/native';
import { createStackNavigator } from '@react-navigation/stack';
import { CartProvider } from './src/context/CartContext';

// Importazione delle schermate grafiche
import CatalogScreen from './src/screens/CatalogScreen';
import DetailScreen from './src/screens/DetailScreen';
import CartScreen from './src/screens/CartScreen';

// TRUCCO PROFESSIONALE: Silenzia gli avvisi di deprecazione di terze parti nel
// terminale dell'esame
LogBox.ignoreLogs(['InteractionManager has been deprecated']);

// Definizione formale di TypeScript per lo Stack di Navigazione
export type RootStackParamList = {
  Catalog: undefined; // Schermata Home: nessun parametro richiesto
  Detail: { productId: number }; // Schermata Dettaglio: richiede obbligatoriamente un ID
  // numerico
  Cart: undefined; // Schermata Carrello: nessun parametro richiesto
};

const Stack = createStackNavigator<RootStackParamList>();

export default function App() {

```

```

return (
  // 1. Avvolgiamo tutto con il CartProvider per rendere il carrello accessibile ad ogni livello
  dello Stack
  <CartProvider>
    <NavigationContainer>
      <Stack.Navigator initialRouteName="Catalog">

        <Stack.Screen
          name="Catalog"
          component={CatalogScreen}
          options={{ title: 'Catalogo Prodotti' }}
        />

        <Stack.Screen
          name="Detail"
          component={DetailScreen}
          options={{ title: 'Dettaglio Prodotto' }}
        />

        <Stack.Screen
          name="Cart"
          component={CartScreen}
          options={{ title: 'Il mio Carrello' }}
        />

      </Stack.Navigator>
    </NavigationContainer>
  </CartProvider>
);
}

```

Fase 4: Avvio dell'Applicazione

Per testare il tutto in modalità pulita svuotando le vecchie memorie cache memorizzate nel computer, digita nel terminale:

npx expo start -c